



SECRETARÍA DE EDUCACIÓN PÚBLICA
TECNOLÓGICO NACIONAL DE MÉXICO



INSTITUTO TECNOLÓGICO DE IGUALA

Nombre de los Alumnos(a):

Carlos Brayan Zuriel Reyes Pita
Jared Manrra Quezada
Miguel Ángel Márquez Meza
Joel Martínez Mena

Carrera:

Ingeniería en Sistemas Computacionales

Semestre:

Sexto Semestre

Asignatura:

Tópicos Bases de Datos Cloud

Ciudad de Iguala; Guerrero

29/05/26

Definición de la estructura de datos (Esquema JSON)

El catálogo de la biblioteca se definió bajo una estructura de arreglo de objetos JSON en el archivo libros.json. Cada entidad "Libro" se modela de forma consistente con las siguientes claves:

- **id:** Identificador numérico único de tipo entero.
- **título:** Cadena de texto con el nombre de la obra.
- **autor:** Cadena de texto con el creador del libro.
- **año:** Año de publicación de la edición.
- **género:** Categoría literaria correspondiente.
- **disponible:** Valor booleano (True/False) que determina el estado de préstamo.

Lógica de Persistencia y Manipulación de Datos

El núcleo de la aplicación desarrollado en Python (app.py) se encarga de mediar entre la interacción del usuario y el archivo físico de datos:

Lectura de registros: Al iniciar la aplicación, se abre el archivo libros.json en modo lectura (r), utilizando la librería nativa json para deserializar el contenido y cargarlo en una lista de diccionarios en memoria RAM. Si el archivo no existe, el sistema atrapa la excepción FileNotFoundError e inicializa una estructura vacía automáticamente.

Operaciones CRUD integradas: El software ofrece funciones específicas para añadir nuevos registros calculando el ID incremental, buscar libros de manera interactiva por su título, actualizar el estado de disponibilidad en tiempo real y eliminar elementos del inventario de forma segura.

Escritura física: Tras cualquier modificación en la lista, el sistema serializa la estructura de datos de vuelta a formato JSON y la escribe en el archivo físico utilizando indentación para mantener la legibilidad de la base de datos NoSQL.

Herramientas del Entorno de Desarrollo

Para la implementación y despliegue local de este proyecto, se requiere un conjunto de herramientas de software estándar:

Python: Lenguaje de programación principal de alto nivel. Destaca por su versatilidad y por incluir de manera nativa la biblioteca json, permitiendo procesar archivos documentales de forma directa sin necesidad de instalar dependencias externas pesadas.

Visual Studio Code (VS Code): Entorno de desarrollo integrado (IDE) ágil y ligero utilizado para la escritura del código fuente, depuración y ejecución directa del script a través de su terminal integrada.

Formato JSON: Estándar abierto basado en texto plano diseñado para el intercambio de datos eficientes, sirviendo como el motor NoSQL documental de almacenamiento local para la persistencia del sistema de biblioteca.

funcionamiento del programa

El programa desarrollado es un gestor de biblioteca con interfaz gráfica en Python. La información de los libros se guarda de forma local en el archivo libros.json mediante TinyDB, una base de datos NoSQL ligera orientada a documentos. La aplicación permite registrar, consultar, modificar, eliminar y visualizar libros desde una ventana creada con Tkinter.

Funcionamiento del código

- Importación de librerías: se utiliza tkinter para crear la interfaz gráfica, ttk y messagebox para componentes visuales y mensajes, y TinyDB junto con Query para administrar los documentos guardados en libros.json.

```
import tkinter as tk
from tkinter import ttk, messagebox
from tinydb import TinyDB, Query

db = TinyDB("libros.json")
Libro = Query()
```

En esta parte se abre la base de datos local llamada libros.json. Si el archivo no existe, TinyDB lo crea automáticamente. La variable Libro se usa para realizar búsquedas y condiciones sobre los registros.

Configuración visual: las variables COLOR_FONDO, COLOR_PANEL y COLOR_BOTON definen los colores principales de la ventana, paneles y botones.

```
COLOR_FONDO = "#eeeeee"  
COLOR_PANEL = "#f7f7f7"  
COLOR_BOTON = "#e1e4e1"
```

Limpieza de campos: la función limpiar_campos borra los datos escritos en el formulario, regresa el estado de disponibilidad a "Sí" y coloca el cursor en el campo ID.

```
def limpiar_campos():  
    entrada_id.delete(0, tk.END)  
    entrada_titulo.delete(0, tk.END)  
    entrada_autor.delete(0, tk.END)  
    entrada_editorial.delete(0, tk.END)  
    entrada_anio.delete(0, tk.END)  
    combo_disponible.set("Sí")  
    entrada_id.focus()
```

Mostrar libros: la función mostrar_libros limpia la tabla, lee todos los documentos almacenados con db.all() y los inserta en el Treeview. También actualiza el total de libros registrados.

```
def mostrar_libros():  
    for fila in tabla.get_children():  
        tabla.delete(fila)  
  
    for i, libro in enumerate(db.all()):  
        tag = "par" if i % 2 == 0 else "impar"  
        tabla.insert("", tk.END, values=(  
            libro.get("id", ""),  
            libro.get("titulo", ""),  
            libro.get("autor", ""),  
            libro.get("editorial", ""),  
            libro.get("anio", ""),  
            libro.get("disponible", "")  
        ), tags=(tag,))  
  
    lbl_total.config(text=f"Total de libros: {len(db.all())}")
```

Guardar libro: la función `guardar_libro` toma los datos del formulario, valida que ID, título y autor no estén vacíos, verifica que el ID no exista y después inserta el documento en `libros.json`.

```
def guardar_libro():
    id_libro = entrada_id.get().strip()
    titulo = entrada_titulo.get().strip()
    autor = entrada_autor.get().strip()
    editorial = entrada_editorial.get().strip()
    anio = entrada_anio.get().strip()
    disponible = combo_disponible.get().strip()

    if id_libro == "" or titulo == "" or autor == "":
        messagebox.showwarning("Aviso", "Llena ID, título y autor.")
        return

    if db.search(Libro.id == id_libro):
        messagebox.showwarning("Aviso", "Ya existe un libro con ese ID.")
        return

    db.insert({
        "id": id_libro,
        "titulo": titulo,
        "autor": autor,
        "editorial": editorial,
        "anio": anio,
        "disponible": disponible
    })
```

Buscar libro por ID: la función `buscar_libro` consulta TinyDB con `db.search(Libro.id == id_libro)`. Si encuentra el registro, coloca sus datos en las cajas de texto para revisarlo o modificarlo.

```
resultado = db.search(Libro.id == id_libro)

if resultado:
    libro = resultado[0]
    limpiar_campos()
    entrada_id.insert(0, libro.get("id", ""))
    entrada_titulo.insert(0, libro.get("titulo", ""))
    entrada_autor.insert(0, libro.get("autor", ""))
```

Búsqueda en tabla: la función `buscar_en_tabla` filtra los registros por coincidencias en el título o el autor. Esto permite localizar libros sin conocer exactamente su ID.

```
libros = db.all()
if texto:
    libros = [l for l in libros if texto in l.get("titulo", "").lower()
or texto in l.get("autor", "").lower()]
```

Modificar libro: `actualizar_libro` verifica que el ID exista y después actualiza los campos del documento seleccionado con `db.update()`.

```
db.update({
    "titulo": entrada_titulo.get().strip(),
    "autor": entrada_autor.get().strip(),
    "editorial": entrada_editorial.get().strip(),
    "anio": entrada_anio.get().strip(),
    "disponible": combo_disponible.get().strip()
}, Libro.id == id_libro)
```

Eliminar libro: `eliminar_libro` valida el ID, solicita confirmación al usuario y elimina el documento con `db.remove()`.

```
if messagebox.askyesno("Confirmar", "¿Eliminar este libro?"):
    db.remove(Libro.id == id_libro)
    messagebox.showinfo("Correcto", "Libro eliminado.")
```

Selección desde la tabla: al hacer clic sobre un libro del catálogo, `seleccionar_libro` toma los valores de esa fila y los carga en el formulario.

```
tabla.bind("<ButtonRelease-1>", seleccionar_libro)
```

Construcción de la ventana: al final del archivo se crea la ventana principal, el formulario, los botones, la barra de búsqueda y la tabla donde se muestran los libros.

```
ventana = tk.Tk()
ventana.title("Gestor NoSQL - Biblioteca")
ventana.geometry("900x680")
```

```
mostrar_libros()
ventana.mainloop()
```

Funcionamiento durante la ejecución

Registrar un libro: El usuario escribe ID, título, autor, editorial, año y disponibilidad. Después presiona Guardar. El libro se añade a libros.json y aparece en la tabla.

The interface consists of a left sidebar with a form titled 'Datos del libro' and a main area with a search bar and a table titled 'Catálogo de libros'.

Datos del libro form:

- ID: 01
- Título: El Principito
- Autor: Antoine
- Editorial: Reynal
- Año: 1943
- Disponible: Sí

Acciones: Guardar, Buscar ID, Modificar, Eliminar, Limpiar, Mostrar todo.

Search bar: Buscar por título o autor: [] [Buscar] [Ver todos]

Catálogo de libros table:

ID	Título	Autor	Editorial	Año	Disp.
01	El Principito	Antoine	Reynal	1943	Sí
02	pavlo	uriel	tekm	2026	Sí

Message: Correcto. Libro guardado. [Aceptar]

Consultar libros: Con Mostrar todo se cargan todos los registros. También se puede buscar por título o autor desde la barra superior de la tabla.

The interface shows a search bar and a table titled 'Catálogo de libros'.

Search bar: Buscar por título o autor: El principit [Buscar] [Ver todos]

Catálogo de libros table:

ID	Título	Autor	Editorial	Año	Disp.
01	El Principito	Antoine	Reynal	1943	Sí

Buscar por ID: Se escribe el ID en el formulario y se presiona Buscar ID. Si existe, los datos aparecen en los campos para consultarlos o modificarlos.

The form titled 'Datos del libro' has the following fields:

- ID: 01
- Título: El Principito
- Autor: Antoine
- Editorial: Reynal
- Año: 1943
- Disponible: Sí

Acciones: Guardar, Buscar ID, Modificar, Eliminar, Limpiar, Mostrar todo.

Total de libros: 2

Modificar un registro: Después de buscar o seleccionar un libro en la tabla, se cambian los datos necesarios y se presiona Modificar. TinyDB actualiza el documento correspondiente.

Datos del libro
ID: 02
Título: cubert
Autor: uriel
Editorial: tekmm
Año: 2026
Disponible: ☐

Buscar por título o autor:

Catálogo de libros

ID	Título	Autor	Editorial	Año	Disp.
01	El Principito	Antoine	Reynal	1943	Sí
02	pavlo	uriel	tekmm	2026	Sí

Correcto

Libro actualizado.

Aceptar

ID	Título	Autor	Editorial	Año	Disp.
01	El Principito	Antoine	Reynal	1943	Sí
02	cubert	uriel	tekmm	2026	Sí

Eliminar un registro: Se escribe o selecciona el ID del libro y se presiona Eliminar. El sistema pide confirmación antes de borrar el registro.

Datos del libro
ID: 02
Título: cubert
Autor: uriel
Editorial: tekmm
Año: 2026
Disponible: ☐

Buscar por título o autor:

Catálogo de libros

ID	Título	Autor	Editorial	Año	Disp.
01	El Principito	Antoine	Reynal	1943	Sí
02	cubert	uriel	tekmm	2026	Sí

Confirmar

¿Eliminar este libro?

Correcto

Libro eliminado.

Aceptar

ID	Título	Autor	Editorial	Año	Disp.
01	El Principito	Antoine	Reynal	1943	Sí

Ver persistencia de datos: Al cerrar y abrir nuevamente el programa, los libros siguen apareciendo porque la información queda guardada en libros.json.

Gestor NoSQL - Biblioteca

BIBLIOTECA NOSQL

Registro y consulta de libros

Datos del libro

ID:

Título:

Autor:

Editorial:

Año:

Disponible:

Sí

Acciones

Guardar

Buscar ID

Modificar

Eliminar

Limpiar

Mostrar todo

Total de libros: 1

Buscar por título o autor:

Buscar

Ver todos

Catálogo de libros

ID	Título	Autor	Editorial	Año	Disp.
01	El Principito	Antoine	Reynal	1943	Sí